Autonomous Software Agents Project Report

Daniele Buondonno
267888
daniele.buondonno@studenti.unitn.it

Sasha Pektovic
264689
sasha.pektovic@studenti.unitn.it

1 Introduction

In this report, we will give an overview of our project for the Autonomous Software Agents course. The project's goal is to create and develop autonomous agents that can play the game Deliveroo.js.
The game consists in picking up and deliverying parcels, which yield different reward values. The game's environment is competitive, as rival agents can be present. The project is divided into two parts.

- BDI Agent: An autonomous agent that uses the Belief-Desire-Intention architecture to observe and perform actions in an environment. This architecture allows the agent to maintain a set of beliefs about the environment obtained through sensing. Through the belief set, the architecture allows the agent to compute possible options (desires) along a utility score for them. The option with the best utility score is selected and performed, if possible.
  The architecture prescribes that a loop is present, where options continue to be generated given new observations, and where the current Intention can be revised.

- LLM Agent: An agent that acts as a partner to the BDI agent. It performs like the BDI agent through a setup composed of API calls and prompts. It must be able to understand the rules of the game and the environment around it.
  It must also be able to understand, evaluate and perform missions delivered to it through the game chat, which can yield positive or negative results.

We will first highlight the structure of the BDI agent, explaining its different components and operations. We will go through the data structures used, the strategies we decided to implement and our reasoning behind both. We will then explain the LLM's agent structure and operations. To conclude we will give some brief results and our thoughts for further improving our solutions.

2 Belief

In the Belief-Desire-Intention architecture, an agent's belief set represents its observations and beliefs over the environment. The term belief is not used casually, as what's 'contained' in its belief set isn't a ground truth, but what the agent has deducted from both present and past observations, which might not be true anymore.
In the Deliveroo game, observations are done through sensing events propagated by the game server. As we will see, in our solution we try to keep the most up to date informations and forget what we confirm to be outdated, so priority is given to the information we're most confident about. We will now go over the different classes implementing the agent's Belief Set.

2.1 Me

This class holds the agent's identity, its position and its score. For the agent's position, we decided to gate non integer values, as they indicate that the agent is attempting to move. This avoids inconsistent information about the position and avoids making wrong assumptions about a succesful move through the environment.

2.2 Parcels

This class holds all the data structures and logic used to represent and update the agent's belief about the parcels it's carrying or that it has observed, during both past and current sensing events. To hold these informations, we use three maps.

2.2.1 Visible

In this map we store all the parcels the agents saw during the latest sensing, indexed by their id. This map is rebuilt every time the agent performs sensing. It is particularly useful to recognise which parcels the agent is most confident about, as it can currently see them.

2.2.2 Known

This map contains all 'known' parcels, indexed by their id. A known parcel is a parcel that has been observed through previous sensings. Along with the parcels data, we store a timestamp indicating when they've been observed. This is particularly useful for computing their utility by estimating their decayed value, if parcel reward decay is currently enabled by the server.
To maintain this data structure as accurate as possible, whenever a known parcel's location is observed, and the parcel is not present, it is removed and forgotten by the agent. If the agent has a partner, this map also holds the information of the parcels reported by the partner.

2.2.3 Carried

This simple map holds the information about the parcels the agent's currently carrying, indexed by their id.

2.3 Crates

This class is specifically designed to represent the agent's beliefs about the crates in the map, if present, and to update such informations. A 'known' map is also used here, it works the same way as for the parcels, only this time it stores crates information. We also made use of a 'byPosition' map as a support structure to make operations easier to code and more efficient, as it indexes crates by their position string.

2.4 Agents

This class holds the agent's beliefs about the other agents perceived in the game environment and the logic to update them. We used a map to store these informations.

2.4.1 Others

This map contains the information about the agents perceived through the latest sensing event, indexed by their id.
During the update, we store their positions regardless if they're an integer or fractional. This is

done on purpose as a fractional coordinate tells the agent that the rival agent is attempting to move, so two tiles are blocked instead of one at that moment.
In fact, if a rival agent is not moving, we only consider its current position as blocked. If it is attempting to move we instead consider its current position and the position its trying to move to as blocked.

2.5 Partner

This class is designed to represent and handle the information about the agent's partner, if there is one. It also holds the information the agent might want to share with its partner.
In particular the two agents share the information about the parcels they're currently carrying, that they're currently observing and the parcel they want to commit to. This helps avoid cases where both agents end up pursuing the same parcel, and it also allows them to evaluate which of the two should pursue a given parcel. The agents can in fact claim a parcel for their own by first evaluating if they're in a better spot to take it compared to the partner.

2.6 World

This class holds the agent's beliefs about the game's world. It stores the data regarding the game's map, configuration and metadata sent out by the server upon connecting or when a change about the config is made. It also contains the set of visible positions observed by the agent throug the latest sensing event.
The most notable data structures in this class are the following.

2.6.1 Parcel Reward Decay Frequency

In order to make precise and useful utlity value computations for both parcel pickups and deliveries, we make use of two things. The decay interval given by the server, stored in milliseconds, and the movement duration, also given by the server through the onConfig sensing event. With these, we compute a 'decay per move' value, which is a good estimate for the reward loss during one movement.
We use the following formula.

$$decayPerMove = \begin{cases} \frac{movementDuration}{decayInterval} & \text{if } decayInterval > 0 \\ 0 & \text{otherwise} \end{cases}$$

2.6.2 VisiblePositions

This set contains all visible positions through the latest sensing event.

2.6.3 Spawners

This map contains all known spawner tiles, a timestamp indicating when they've been last observed, and a boolean indicating if an operational delivery tile can be reached from each one of them. These values are indexed by a string indicating their coordinates.
An operational delivery tile is one that, when reached, doesn't lead to a deadlock. We added this boolean to deal with edge cases in some maps where some spawner or delivery tiles led to a deadlock once reached. This makes the agent avoid going to deliveries or spawners that would end its match.

2.6.4 Deliveries

This map is the equivalent of the 'Spawners' map, but it holds information about the known delivery tiles, paired with a boolean indicating if an operational spawner can be reached from there.

3 Desires and Intention Loop

As prescribed by the BDI architecture, the agent consults its beliefs and computes a set of desires, along with a utility value representing their potential reward if accomplished. The best option, that is the one with the highest utility value, is chosen and it becomes the current intention.

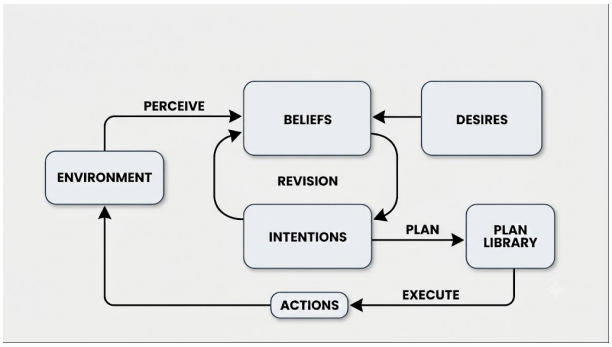

Figure 1: The BDI Loop.

3.1 Desires Generation

The generation of desires makes use of the agent's current and past observations. The partner's shared information is also considered.
During generation, the desires are stored in an array containing objects representing the desires and the information needed to perform, evaluate and compare them.
These objects contain the following fields:

- type: it identifies the action to be performed by a given desire. It's a simple string of the form 'go_pick_up' or 'go_deliver'.
- target: the coordinates (x,y) of the desire's destination.
- utility: the utility value computed for the desire.

We decided to develop a small, but efficient number of possible desires. We defined the following types of desires.

- go_pick_up: it defines the desire of picking up a parcel.
- go_deliver: it defines the desire of deliverying the currently carried parcels.
- go_to_spawner: this type of desire is used as an exploration option. It is designed to be generated only when the agent currently doesn't have the option of either deliverying or picking up parcels.

3.1.1 Distances

To compute the utility values of our desires, we take into account the distances to reach the desires' target tiles. This is important as the game server can enable a parcel reward decay over time. This must be taken into account when evaluating which desire to perform, as further targets lead to likely smaller rewards as the parcels the agent is carrying and the parcels on the ground progressively lose more and more value.
To compute these distances, we've made use of the Breadth First Search (BFS). Since all actions costs are equal, this search yields optimal results, making the obtained distances the shortest path distances to a destination. We further justify this choice over other options such as heuristical search algorithms in a later section of this report.

3.1.2 Pick-up Utility

This desire can be generated when the agent knows about at least one parcel that isn't currently being carried by anyone.
To generate this desire, we compute an utility score for each batch of parcels, a batch can be composed by only one parcel. We don't exclude this computation to a single parcel as multiple parcels can lay on a single tile, forming a batch whose utility score must be a sum of the utility of all its parcels. In addition, a pickup action picks up all the parcels in a given tile.
To compute the utility value of batches, we decided to also take into account the distance to deliver the picked up parcels from their location.
The distance used is the following.

$$totalDistance = distanceToBatch + distanceAfterPickup$$

Where 'distanceAfterPickup' is the distance to the closest delivery tile with respect to the batch's location.
This allows the agent to have an accurate and complete estimation of the whole process, it effectively gives it a prediction of the potential reward for picking up and then delivering one or more parcels. It also allows the agent to have a good measurement of comparison for utility of all the batches it can consider to pickup.
A batch's pickup utility score is given by the following formula.

$$batchPickupUtility = \sum_{p \in batch} \max(0, p.reward) - (decayPerMove * totalDistance)$$

It computes an estimation of the batch's reward once it has been picked up and the delivery tile is reached, taking decay into account.
To compute the pickup action's utlity, we also take into account the estimated reward of the parcels the agent is currently carrying, as their reward is gonna decay throughout the whole process.
The carried parcels' estimated reward is the following.

$$carriedReward = \sum_{p \in carried} \max(0, p.reward) - (moveDecay * distanceToDelivery)$$

For each pickup action option utility, we then use this formula.

$$pickupUtility = \frac{carriedReward + batchPickupUtility}{\max(1, totalDistance)}$$

The pickup option that gets inserted into the desires array is the one with the highest utility value.

3.1.3 Put-down Utility

This desire can be generated if the agent is currently carrying at least 1 parcel.
To generate this desire, we scan through all the delivery tiles in the game map. For each delivery tile, we compute the shortest path's distance to it, if it's reachable. If the given delivery tile is reachable, we compute its utility value.
We compute the delivery utility with the following formula:

$$deliveryUtility = \frac{expectedDeliveryReward}{\max(1, distanceToDelivery)}$$

Where 'expectedDeliveryReward' is the expected reward for delivering the parcels currently carried by the agent taking distance and reward decay into account. To compute this expected reward, we iterate through all carried parcels and compute an estimation of their decayed reward value once the agent reaches the delivery tile.
The formula is the following.

$$expectedDeliveryReward = \sum_{p \in carried} \max(0, p.reward) - (moveDecay * distanceToDelivery)$$

3.1.4 Exploration Utility

This desire is generated only when there is no other desirable action for the agent to perform. This happens when it has no parcels to deliver or to pickup, so the only option is to explore towards spawner tiles currenlty out of vision.
To compute the utility value of this desire, we iterate through the out of vision spawners, giving priority to operational ones. We then compute their exploration utility value.
This value is computed with the following formula.

$$spawnerExplorationUtility = movesSinceCheck / \max(1, Distance)$$

Where 'movesSinceCheck' is the amount of moves since the last time the agent saw the given spawner tile. This allows the agent to prioritise spawners it hasn't checked recently, as it's likely their state has changed.

3.1.5 Multipliers

The game server can assign missions to individual agents, teams or all players. These missions can assign reward multipliers for delivering stacks of parcels or for delivering in a certain delivery tile, among other cases. These multipliers are applied if needed after the utility values are computed and before they're inserted into the desires array.

3.2 Intention Loop and Intention Revision

3.2.1 Intention Loop

The intention loop serves as the continuous execution cycle of the BDI agent.
Upon receiving a sensing event from the game server, the agent updates its Beliefs. Next, it computes the utility values for all possible desires using the formulas established previously. The desire yielding the highest utility score is selected as the agent's current Intention.
The agent has to then establish the plan, that is, the sequence of actions to perform to fulfill the intention. To do this, the agent either utilises a PDDL solver or establishes a path through a BFS search. We will explain this decision in a later section of this report.

3.2.2 Intention Revision

Because the game environment is highly dynamic and competitive, blind commitment to a single plan is unfeasible. Therefore, the agent continuously evaluates its active intention against incoming sensings and game events, this process is the Intention Revision. Intention revision is triggered under two primary conditions:

- Impossibility: The current intention is no longer viable. For example, if a rival agent picks up the target parcel, or if the path to a delivery tile becomes completely blocked by other agents, the current plan is instantly invalidated.
- Suboptimality: A newer, significantly better opportunity arises. If a high-value parcel spawns nearby, the newly computed utility of picking it up might drastically outweigh the utility of the agent's current intention.

When a revision is triggered, the agent drops its current plan, adopts the new optimal desire as its intention, and generates a new plan to pursue it. This ensures the agent remains both goal-oriented and highly reactive to the evolving state of the game.

4 Planning

Once an intention is established by the agent, the next step is planning its execution. A plan is a sequence of steps that concludes with the fulfillment of the plan's objective.
To establish a plan, we have different options. The first is the more traditional solution for artificial intelligence planning found in the PDDL language. Given a domain, with a set of rules, states and possible actions for all the states, it allows to search and find a plan that satisfies a given task, if it's possible.
The second option is less traditional and more standard programming oriented, as we simply make use of a search algorithm to find a path to a desire's target and execute its action.
We decided to use PDDL only when strictly needed, while we 'defaulted' to using a BFS search for all other cases. We will explain in the following sections our motivations and use of them.

4.1 BFS Search and Planning

For all the cases where PDDL is not strictly needed we opted to use our version of a BFS search. The Breadth First Search (BFS) is a classic search algorithm that can be used with graphs or grid like environments. It performs a search by expanding all nodes in the current depth. Once every node in the current depth has been explored, the search proceeds to the next depth. This process continues until the goal is found. As mentioned earlier, this search is optimal for domains where the costs for all actions are equal. This allowed us to use our BFS algorithm to compute shortest path distances for better utility value computations.
Once an intention is established, the agent uses the BFS search to find the shortest path to the intention's target. Once the path is found, the agent follows it and, once it has reached the target destination, performs the action.
Using this search, the BDI agent ended up being very fast and efficient. This is not the case for PDDL planning. Also, we note that this solution is much more flexible to intention revision, or changes in the environment that require a different path to be taken to fulfill the current intention. This is due to the fact that computing another BFS search is much faster than going through the PDDL solver.

4.2 PDDL

We used PDDL for planning only in cases where it was strictly needed, as its performance is greatly worse than the one we implemented through a simple search.
The slower performance, especially in our case, is given by the fact that to find a plan, PDDL goes through its own solver. This process is slower than simply performing a search.
We decided to use PDDL for maps with crates. Specifically, when an agent encounters a crate in its path it calls the PDDL solver to find a path that allows the agent to move towards its current intention's target. In these cases, the agent stops for some seconds, waiting for the solver to receive the problem's details and for it to send back the satisfying plan. Given that BFS alone couldn't solve this kind of problem, we find that relegating PDDL for this use case alone is a satisfying solution, as the agent performs very efficiently in the majority of game worlds.
We note that PDDL is also less flexible to changes in the environment or intention revision. If a path needs to be redefined due to changes in the environment, such as new obstacles, or because of a change of the current intention, the agent has to sit and wait for the whole process again.

5 LLM Agent and Communication

5.1 LLM Agent Architecture

The LLM Agent operates as a complementary, reasoning partner alongside the BDI agent. Rather than relying solely on hardcoded utility formulas and planning, the LLM agent leverages OpenAI's API to interpret game states, evaluate natural-language events, and decide optimal actions. It is also able to interpret, evaluate and perform missions given to it through the in game chat. It can also communicate with the BDI agent. The LLM Agent's architecture performs these operations in order to function.

1. State Serialization and Prompting: On each sensing tick the agent serializes its localized belief state, including grid coordinates, carried parcel rewards, visible decay rates, and map obstacles into a structured prompt context.

2. Action Reasoning: The system prompt embeds Deliveroo.js game rules, movement constraints, and action formats. The LLM outputs structured responses, indicating its own reasoning and computations, if needed. It then performs the required actions to fulfill its own intentions.

3. Mission Evaluation: As mentioned earlier, missions can be assigned from the server via game chat. They present side objectives with variable positive or negative rewards. The LLM agent parses these natural language announcements, evaluates the risk versus reward ratio based on its and the game's current status, and decides whether to accept, delegate, or ignore the mission.

4. Robustness and Fallback: To handle API latency or invalid LLM generations, the agent validates output paths using the BFS/PDDL planner before execution, falling back to basic deterministic actions if a timeout occurs.

5.2 Multi-Agent Communication Protocol

The BDI agent and the LLM Agent form a team. They communicate with each other, adding information to each other's belief and share other informations.
The LLM agent can let the BDI agent know about some ongoing mission, active multipliers or potential negative rewards for specific actions. They also let each other know about their

intentions, to avoid edge cases where both pursue the same objective.
Some missions may require strict cooperation between the two agents. The LLM agent is able to send simple instructions to the BDI agent, through either target coordinates or simple instructions. We also implemented a simple functionality that lets them choose which of the two should perform an intention and pick the one that is in the better position to efficiently perform it.
To maximize overall team utility and prevent redundant efforts, the BDI and LLM agents communicate asynchronously using the game server's socket conncection.

6 Challenge Benchmarks and Conclusions

[Present the results of the project.]